COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/21/2026	**DATE SUBMITTED:**	07/22/2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

HANDS-ON ACTIVITY #3.1 **Creating a Simply Linked List**

1. PROCEDURE OUTPUT

Codes:
simply_LL.h

```cpp
#ifndef SIMPLY_LL_H
#define SIMPLY_LL_H
#include <iostream>
template <typename T>
class SimplyList{
    public:
    T data;
    SimplyList<T> *next = 0;
};
/*ListTraversal (parameter: pointer to node n)
//WHILE n IS NOT EQUAL TO null
//PRINT data OF n
//GO TO NEXT NODE n := next
//ENDWHILE
//PRINT next line
//END

traversal*/
template <typename T>
void listTraversal(SimplyList<T>* head){
    if(head == 0){
        std::cout << "List is empty" << std::endl;
        return;
    }

    while(head != NULL){
        std::cout << head->data;
        if(head->next != 0){
            std::cout<<"->";
        }
        head = head->next;
    }
    std::cout<<std::endl;
}


template <typename T>
void sllinsertHead(T newData, SimplyList<T>** currentHead){
    //1. Allocate memory for the new node
    SimplyList<T>* newNode = new SimplyList<T>;
    //2. Put our data into the new node
    newNode->data = newData;
    //3. Set next of the new node to point to the previous head
    newNode->next = *currentHead;
    //4. Reset head to the point to the new node
    *currentHead = newNode;
}
```

```
48  template <typename T>
49  void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
50      if(prevNode == 0)
51      {
52          std::cout << "Previous Node cannot be NULL" << std::endl;
53          return;
54      }
55      SimplyList<T>* newNode= new SimplyList<T>;
56      newNode->data = newData;
57      newNode->next = prevNode->next;
58      prevNode->next = newNode;
59  }
60
61  template <typename T>
62  void sllInsertEnd(T newData, SimplyList<T>** head){
63      //1. Allocate new node
64      SimplyList<T>* newNode = new SimplyList<T>;
65      //2. Dereference to the new node
66      SimplyList<T>* currentNode = *head;
67      //3. Store data in the new node
68      newNode->data = newData;
69      //4. Point next of new mode to NULL
70      newNode->next = 0;
71      //5. Traverse the list until the next of the node is null
72      while(currentNode->next != 0){
73          currentNode = currentNode->next;
74      }
75      //6. Point the next of the current mode to the new node
76      currentNode->next = newNode;
77  }
78
79  template <typename T>
80  void sllDeleteNode(T findData, SimplyList<T>** head){
81      //check if there is no list to be deleted
82      if(*head == 0) return;
83
84      SimplyList<T>* currNode = *head;
85      SimplyList<T>* prevNode = 0;
86
87      //searching loop
88      while(currNode != 0 && currNode->data != findData){
89          prevNode = currNode;
90          currNode = currNode->next;
91      }
92
```

```
93      //data was not found:
94      if (currNode == 0) return;
95
96      //what if the node to be deleted is the head node?
97      if (prevNode == 0){
98          *head == currNode->next;
99      }else{
100         prevNode->next = currNode->next;
101     }
102     //we have to delete the node?
103     delete currNode;
104 }
105 template <typename T>
106 void sllDeleteList(SimplyList<T>** head){
107     SimplyList<T>* current = *head;
108
109     while(current != 0){
110         SimplyList<T>* temp = current;
111         current = current->next;
112         delete temp;
113     }
114     *head = 0;
115 }
116 #endif
```

main.cpp:

```cpp
#include <iostream>
#include "simply_LL.h"

int main(){

// creating an empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second = new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;
    SimplyList<char>* fifth = new SimplyList<char>;


//assign the data:
    head->data = 'J';
    head->next = second;

    second->data = 'I';
    second ->next = third;

    third->data = 'M';
    third->next = fourth;

    fourth->data = 'Q';
    fourth->next = NULL;

    std::cout<< "Original list: " <<std::endl;
    listTraversal(head);

    std::cout<< "\nInserting a new node at the head: " << std::endl;
    SimplyList<char>* newNode = new SimplyList<char>;
    sllinsertHead('Q', &head);
    listTraversal(head);

    std::cout<< "\nInserting a new node at the third: " <<std::endl;
    sllGeneralInsert('P', head->next->next);
    listTraversal(head);

    std::cout<< "\nInserting a new node at the END: "<<std::endl;
    sllInsertEnd('U', &head);
    listTraversal(head);

    std::cout<<"\nDeleting the node: " <<std::endl;
    sllDeleteNode('X', &head);
    listTraversal(head);

    std::cout<<"\nDeleting the all the nodes: " << std::endl;
    sllDeleteList(&head);
    listTraversal(head);


return 0;

}
```

Output of the code:

```
Original list:
J->I->M->Q

Inserting a new node at the head:
Q->J->I->M->Q

Inserting a new node at the third:
Q->J->I->P->M->Q

Inserting a new node at the END:
Q->J->I->P->M->Q->U

Deleting the node:
Q->J->I->P->M->Q->U

Deleting the all the nodes:
List is empty

--------------------------------
Process exited after 0.01434 seconds with return value 0
Press any key to continue . . .
```

Analysis:

In the header file the code was given and include guard which are #ifndef, #define and #endif. #ifndef is used if the macro is not defined, #define was used if its not macro and #endif ends it. In the first code(simply_LL.h), we used template <typename T> as our parameter. The code given then make a class for SimplyList which contains a public class T data so we can the value of the node and SimplyList<T>* next as our pointer. The code that was given now used a template but this time its for listTraversal using void, the SimplyList<T>* head is our first node which is also alongside with the templates like list sllInsertHead, sllGeneralInsert, sllInsertEnd, sllDeleteNode and sllDeleteList. Back to the traversal we used while if n is not equal to null and also used the -> as a pointer, if head is equal to null (0) it would print “List is empty.”. In the sllInsertHead, asides from what I said about the listTraversal and transferring the node, it would just pointer to the currentHead using -> and make *currentHead as a new node. In the sllGeneralInsert, we used the prevNode (previousNode) and newData, similar to sllInsertHead it would print that the “Previous node cannot be NULL” if the prevNode is 0, it would point until to the newNode. In the sllInsertEnd, its similar to the two but it would traverse the list until it make it to null, the will point the newNode as the pointer. In the sllDeleteNode, it would check if the list is not deleted, then it would search for a loop if it doesn’t find any data, if the node to be deleted is the head node or the whole node itself. In the sllDeleteList, as the pointer stores it would enter a while loop, it would continue until there are nodes left, in the while loop the pointer will be stored in a temporary variable which is the current, then it would be to the next

node which is current->next then it will now delete the temporary variable (temp) which store the node which also makes the *head a nullptr (0). In the main cpp file, asides the header file #include <iostream>, the code also include #include “simply_LL.h” as the linked list file. Now in the main function, we create an empty list based from the head until the last(or fifth). The code would assign the data by the use of -> which the code also includes the templates from the other file to be printed.

Conclusion:
In this lesson, I barely absorbed the lesson because of how many templates and nodes that were in the lesson. I was blown away from the lesson since it made me think about my life decisions. Though I may understand how the nodes work, I did a few modifications to make the code work in C++. Overall, this lesson proves how much I will still need to learn.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT to generate ideas on how I’ll explain the code in some of the templates that were given as well as the purpose of including guards.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.